SaTTric+ JSON Payload Transfer Specification

1. Scope

This document specifies exactly how configuration JSON (Section 4-8 of the UI Developer Guide) travels from the UI, through the backend, to the physical device — and back. It covers two transport paths that carry the **same JSON schema**:

- **Cloud path** — UI → Backend REST API → device over GPRS/MQTT (normal operation)
- **Local path** — Commissioning tool → device directly over the spare UART (factory setup, or sites with no cellular coverage yet)

The device itself never parses raw JSON — it only understands the compact binary frame format defined in the companion Protocol & Firmware Design document. Translating JSON into binary frames is the backend's job (cloud path) or the commissioning tool's job (local path). This document specifies both sides of that boundary.

2. Cloud path — REST API

2.1 Base URL and versioning

`https://api.sattric.io/v1`

The `v1` path segment is the API version. Breaking changes to request/response shapes increment this; additive fields do not.

2.2 Authentication

Header	Value	Notes
`Authorization`	`Bearer <token>`	OAuth2 token or device-scoped API key, issued per user/organization
`Content-Type`	`application/json`	All requests and responses
`Idempotency-Key`	UUID	Required on all `POST` requests — see Section 2.6

A request without a valid `Authorization` header receives `401 Unauthorized`. A token valid for authentication but lacking permission on the target device receives `403 Forbidden`.

2.3 Request envelope

Every request body follows the same outer shape, regardless of which config area it targets:

```
{
  "schema_version": "1.0",
  "request_id": "b3f1c2a0-...-uuid",
  "data": { }
}
```

Field	Type	Meaning
schema_version	string	JSON schema version the payload conforms to
request_id	string (UUID)	Generated client-side, echoed back in the response, useful for support/debugging
data	object	The actual config payload — shape depends on the endpoint (Section 3)

2.4 Response envelope

```
{
  "schema_version": "1.0",
  "request_id": "b3f1c2a0-...-uuid",
  "status": "success",
  "data": { "config_version_draft": 14 },
  "error": null
}
```

On failure, `status` is `"error"`, `data` is `null`, and `error` is populated:

```
{
  "schema_version": "1.0",
  "request_id": "b3f1c2a0-...-uuid",
  "status": "error",
  "data": null,
  "error": {
    "code": "ERR_RANGE",
    "message": "raw_high must be greater than raw_low",
    "field": "data.calibration.raw_high"
  }
}
```

2.5 HTTP status codes

Code	Meaning
200 OK	Read succeeded, or commit/rollback succeeded
202 Accepted	Config staged, not yet committed
400 Bad Request	Malformed JSON or failed schema validation
401 Unauthorized	Missing/invalid auth token
403 Forbidden	Token valid but lacks permission on this device
404 Not Found	Device ID doesn't exist or isn't provisioned to this account
409 Conflict	`config_version` in the request doesn't match the device's current version (someone

Code	Meaning
	else edited it — see 2.7)
422 Unprocessable Entity	Passed schema validation but failed semantic validation (e.g. two channels both claim channel_id 3)
503 Service Unavailable	Device is offline; request has been queued (cloud path only — see 2.8)

2.6 Idempotency

Every `POST` must carry an `Idempotency-Key` header (a client-generated UUID). If the same key is replayed (e.g. the UI retries after a timeout without knowing whether the first attempt succeeded), the backend returns the original response instead of applying the change twice. Keys are valid for 24 hours.

2.7 Optimistic concurrency

`GET` responses for any config area include the device's current `config_version`. When submitting a change, the UI includes the version it was editing against:

```
{
  "schema_version": "1.0",
  "request_id": "...",
  "base_config_version": 13,
  "data": { }
}
```

If the device's actual version has moved on since (someone else committed a change), the backend returns `409 Conflict` with the current version in the error body, and the UI should re-fetch and let the user reconcile — the same pattern as editing a shared document.

2.8 Offline devices

If a device is not currently connected to the cloud (GPRS link down, or unpowered), the backend still accepts staged `POST` requests, queues them, and returns `503 Service Unavailable` with a `Retry-After` hint and a queued-request ID. The UI should show a "changes saved — will apply when device reconnects" state rather than treating this as a failure. `COMMIT_CONFIG` requests are queued the same way.

3. Endpoint catalog

Method	Path	Purpose
GET	/devices/{id}	Device info, firmware version, current config_version
GET	/devices/{id}/config/meter	Read current primary meter register map
POST	/devices/{id}/config/meter	Stage a full register map

Method	Path	Purpose
		(array, see 3.1)
GET	/devices/{id}/config/secondary-modbus	Read auxiliary bus config
POST	/devices/{id}/config/secondary-modbus	Stage auxiliary bus config
GET	/devices/{id}/config/io-channels	Read all 8 channel configs
POST	/devices/{id}/config/io-channels/{channel_id}	Stage one channel's config
POST	/devices/{id}/config/io-channels/{channel_id}/calibration	Stage calibration only
GET	/devices/{id}/config/outputs	Read all output configs
POST	/devices/{id}/config/outputs/{output_id}	Stage one output's config
GET	/devices/{id}/config/uart-ports	Read UART port assignments
POST	/devices/{id}/config/uart-ports/{port_id}	Stage one port's config
GET	/devices/{id}/config/rules	Read all event rules
POST	/devices/{id}/config/rules	Stage a new rule
PUT	/devices/{id}/config/rules/{rule_id}	Stage an update to an existing rule
DELETE	/devices/{id}/config/rules/{rule_id}	Stage removal of a rule
GET	/devices/{id}/config/draft	Full diff of staged-but-uncommitted changes
POST	/devices/{id}/config/commit	Atomically activate all staged changes
POST	/devices/{id}/config/rollback	Revert to last committed configuration
GET	/devices/{id}/tags	Current live values from the Tag Database (polling)
WS	/devices/{id}/stream	Live tag value push (WebSocket, see 3.2)
GET	/devices/{id}/events	Rule-trigger and alarm history

3.1 Example: staging a meter register map

```
POST /devices/SAT-2201/config/meter
Idempotency-Key: 6f2e...
{
  "schema_version": "1.0",
  "request_id": "req-001",
  "data": {
    "bus_id": 0,
    "slave_address": 1,
    "baud": 9600,
    "parity": "none",
    "stop_bits": 1,
    "register_map": [
      { "parameter": "Voltage_L1", "modbus_function":
"READ_HOLDING_REGISTERS",
        "start_register": 3027, "register_count": 2, "data_type":
"FLOAT32",
        "byte_order": "BIG_ENDIAN", "scale_factor": 1.0, "offset":
0.0,
        "poll_interval_ms": 2000 },
      { "parameter": "Current_L1", "modbus_function":
"READ_HOLDING_REGISTERS",
        "start_register": 3007, "register_count": 2, "data_type":
"FLOAT32",
        "byte_order": "BIG_ENDIAN", "scale_factor": 1.0, "offset":
0.0,
        "poll_interval_ms": 2000 }
    ]
  }
}
```

Response:

```
{
  "schema_version": "1.0",
  "request_id": "req-001",
  "status": "success",
  "data": { "config_version_draft": 14, "entries_accepted": 2 },
  "error": null
}
```

The UI sends the **whole register map array in one call** — up to roughly 100 entries comfortably fits under typical body-size limits. The backend, not the UI, is responsible for splitting this into multiple binary frames when relaying to the device (see Section 4). This is the key simplification the JSON layer buys: the UI never needs to know about the device's 240-byte-per-frame limit (set by its 256-byte receive buffer).

3.2 Live values: polling vs. streaming

- `GET /devices/{id}/tags` — simple polling, returns a snapshot:

```
{ "schema_version": "1.0", "data": {
    "tags": [
      { "tag": "Voltage_L1", "value": 231.4, "unit": "V",
"timestamp": "2026-08-15T10:22:01Z" },
      { "tag": "Tank_Level_Pct", "value": 62.0, "unit": "%",
"timestamp": "2026-08-15T10:22:03Z" }
    ] } }
```

- `wss://api.sattric.io/v1/devices/{id}/stream` — WebSocket, pushes the same `{tag, value, unit, timestamp}` objects as they update. Use this for a live dashboard screen instead of polling on a tight interval.

4. How the backend turns JSON into device frames

This section is for the backend team's awareness — a UI developer does not implement this, but understanding it explains why the cloud path has no payload-size limit that matters to the UI.

1. The backend validates the JSON against a formal JSON Schema for that endpoint (Section 6).
2. Each array-type payload (register map entries, rule lists) is split into groups sized to fit the device's 240-byte binary frame payload limit — 12 register-map entries per frame at the current entry size, driven by the device's 256-byte receive buffer.
3. Each group becomes one `SET_METER_MAP` (or equivalent) binary frame, sent in sequence over the device's active transport (GPRS/MQTT session or local UART).
4. The device ACKs each frame individually; the backend only reports success to the UI once every frame in the group has been ACKed. A NACK on any frame fails the whole staged request and is surfaced to the UI as `422` with the specific entry that failed.
5. `COMMIT_CONFIG` is sent as a single binary frame once all staging frames for that request succeeded.

5. Local path — direct serial commissioning

For factory commissioning or offline sites, a local tool talks to the spare UART directly, with no backend in between. It carries the identical JSON payloads from Sections 3 and 7 of the UI Developer Guide, but needs its own framing and chunking scheme, since there's no backend service left to do it:

5.1 Local frame format

```
[4-byte length prefix, LE][JSON payload bytes][4-byte CRC32]
```

One JSON object per frame, newline-free (the length prefix makes delimiters unnecessary). Maximum JSON payload per frame: 4096 bytes.

5.2 Chunking large arrays locally

When a register map or rule list is too large for one 4096-byte frame, the local tool splits it itself, using a batch envelope:

```
{
  "schema_version": "1.0",
  "batch_seq": 1,
  "batch_total": 3,
  "data": { "register_map": [ ] }
}
```

The device buffers batches by `batch_seq` and only runs validation once it has received `batch_total` batches for that config area. If a batch is missing or fails CRC, the device NACKs with the missing `batch_seq`, and the tool resends just that batch.

5.3 Everything else is identical

Commit, rollback, config_version, and the per-config-area JSON shapes are exactly the ones defined in Sections 3 and 6 — only the outer framing differs between the two transport paths. This is deliberate: a config JSON file exported from one path can be replayed through the other without modification.

6. JSON Schema references

Formal JSON Schema (draft 2020-12) definitions are maintained alongside the API and published at `https://api.sattric.io/v1/schemas/{area}.json` for each config area (`meter`, `io-channel`, `output`, `uart-port`, `rule`). UI developers should validate forms client-side against these schemas before submitting, so validation errors surface immediately in the form rather than as a round-trip `400/422` response.

Example — abbreviated schema for one register-map entry:

```
{
  "type": "object",
  "required": ["parameter", "modbus_function", "start_register",
"register_count", "data_type"],
  "properties": {
    "parameter": { "type": "string" },
    "modbus_function": { "enum": ["READ_HOLDING_REGISTERS",
"READ_INPUT_REGISTERS"] },
    "start_register": { "type": "integer", "minimum": 0, "maximum":
65535 },
    "register_count": { "type": "integer", "enum": [1, 2, 4] },
    "data_type": { "enum": ["INT16", "UINT16", "INT32", "UINT32",
"FLOAT32", "BCD"] },
    "byte_order": { "enum": ["BIG_ENDIAN", "LITTLE_ENDIAN",
"WORD_SWAPPED"] },
    "scale_factor": { "type": "number" },
    "offset": { "type": "number" },
    "poll_interval_ms": { "type": "integer", "minimum": 100,
"maximum": 60000 }
  }
}
```

7. Size limits at a glance

Layer	Limit	Enforced by
Cloud JSON request body	256 KB	Backend API gateway
Register map array (cloud)	~100 entries per call (practical, not hard-capped)	Backend, chunked transparently
Local JSON frame payload	4096 bytes	Local commissioning tool + device
Device binary frame payload	240 bytes	Firmware — 256-byte receive buffer minus frame overhead (see Hardware Bridge Protocol Spec)
Rule count	64 rules per device	Firmware config storage
I/O channels	8 (fixed by hardware)	Firmware
Outputs	8 relays + 4 PWM (fixed by hardware)	Firmware

The takeaway for the UI team: **design forms and API calls against the cloud JSON limits (generous), not the device's binary frame limits (tight)** — the translation layer exists specifically so you don't have to think about the tighter numbers.